Hitesh Rathore(HR) Notes
LINKEDIN LINK:
https://www.linkedin.com/in/hitesh-rathore-835279305/

JS TYPE CONVERSION IN JAVASCRIPT

FIRST PRINCIPLES • VISUAL NOTES

WHAT IS TYPE CONVERSION?

Changing a value from one data type to another.

"10" (String) → 10 (Number)

WHY DOES JAVASCRIPT NEED TYPE CONVERSION?

JS must decide how to treat values in operations.

"10" + 5 ??
15 or "105"

TWO TYPES OF CONVERSION

① Implicit (Automatic)
JS does it itself.
"5" * 2
↓
Number("5") → 5
↓
5 * 2 → 10

② Explicit (Manual)
Developer does it.
Number("5")
↓
5

STORY TIME

Hitesh wants to add "5" + 2

① Hitesh writes "5" + 2 → "5" + 2

② JS thinks: One is string, one is number. What should I do?

③ For + operator, JS prefers string concatenation. "5" + "2"

④ Converts 2 to "2" and joins. "5" + "2" = "52"

Result → "52"

COMMON CONVERSIONS

String → Number	
Number("123")	123
Number("12.5")	12.5
Number("abc")	NaN
Number("")	0
Number(" ")	0

Whitespace is removed first.

Number → String	
String(100)	"100"
String(true)	"true"
String(null)	"null"
String(undefined)	"undefined"

Converts value to its text form.

String → Boolean	
Boolean("")	false
Boolean("Hello")	true
Boolean("false")	true
Boolean("0")	true

Only empty string is false.

Number → Boolean	
Boolean(0)	false
Boolean(-0)	false
Boolean(0n)	false
Boolean(NaN)	false
Boolean(100)	true
Boolean(-5)	true

0, -0, 0n, NaN are false.

Null Conversions	
Number(null)	0
String(null)	"null"
Boolean(null)	false

null is absence of any value.

Undefined Conversions	
Number(undefined)	NaN
String(undefined)	"undefined"
Boolean(undefined)	false

undefined is uninitialized value.

TRUTHY & FALSY

Falsy Values (Only these)
- false
- 0
- -0
- 0n
- "" (empty string)
- null
- undefined
- NaN

All these are treated as false in boolean context.

Everything else is Truthy
- "hello"
- []
- {}
- function(){}
- Infinity
- -1
- 100
- "0"
- new Date()

All these are treated as true in boolean context.

Meaning
Falsy: Values that are treated as false in boolean context.
Truthy: Values that are treated as true in boolean context.

Example

```
if ("") {
  // Won't run
}
```

```
if ([]) {
  // Will run
}
```

NaN – FULL EXPLANATION

NaN = Not a Number

① What is NaN?
NaN is a special value used to represent an invalid number result.
Type:
number
typeof NaN → "number"

② When is NaN produced?
- Number("abc")
- "abc" - 1
- 0 / 0
- Infinity - Infinity
- parseInt("abc")
- Math.sqrt(-1)

③ Important Facts
NaN is not equal to anything, even NaN itself.
NaN == NaN → false
NaN === NaN → false
- isNaN(NaN) → true
- Number.isNaN(NaN) → true

NaN means:
"This operation doesn't return a valid number."

KEY TAKEAWAYS

- ✓ JS converts values automatically (implicit).
- ✓ You can convert manually (explicit).
- ✓ Know falsy values – only 8 of them.
- ✓ Everything else is truthy.
- ✓ NaN means invalid number result.
- ✓ Truthy & Falsy used in conditions, loops, defaults, validations, and more.

Remember This Mantra
If it feels empty → Falsy
If it has value → Truthy

SPECIAL OPERATION CASES

①	②	③	④	⑤	⑥	⑦	⑧
"5" + 2	"5" - 2	"5" * "2"	"5" / "2"	true + 1	false + 1	null + 1	undefined + 1
"52"	3	10	2.5	2	1	1	NaN
+ prefers concatenation	- requires numbers	Both become numbers	Both become numbers	true → 1	false → 0	null → 0	undefined → NaN

DOUBLE EQUALS (==)
Allows conversion
5 == "5"
↓
true
"5" becomes 5

TRIPLE EQUALS (===)
No conversion
5 === "5"
↓
false
How === works
① Data Type (Strict Check)
② Value (If types are same)
Different types → false

NOTE: == allows type conversion before comparison.
=== does not allow conversion.

NOTE

Null (==) and Undefined (==) are equal to each other, but none of them is equal to any other value.

null == undefined → true

Neither null nor undefined is equal to any other value (number, string, boolean, object, etc.).

1 BASIC COMPARISONS

Expression	Explanation	Result
null == undefined	Null is loosely equal to undefined	true
null === undefined	Null is not strictly equal to undefined because data types are different	false
null == 0	Null is not equal to any number	false
null == ""	Null is not equal to any string	false
null == false	Null is not equal to any boolean	false
null == []	Null is not equal to any object/array	false
null == {}	Null is not equal to any object	false
undefined == 0	Undefined is not equal to any number	false
undefined == ""	Undefined is not equal to any string	false
undefined == false	Undefined is not equal to any boolean	false
undefined == []	Undefined is not equal to any object/array	false
undefined == {}	Undefined is not equal to any object	false

KEY POINT

Only these two are loosely equal (==):

null == undefined → true

Every other comparison with null or undefined is false.

2 WHY DOES null == undefined RETURN true?

- In JavaScript, == (loose equality) performs type conversion before comparison.
- Both null and undefined are considered "empty" or "absence of value".
- According to the ECMAScript specification, they are treated as equal in loose comparison.
- But === (strict equality) compares both value and type — and they are different types.

```
typeof null        // 'object' (quirk in JS)
typeof undefined   // 'undefined'

null == undefined   // true
null === undefined  // false
```

KEY TAKEAWAY
null == undefined → true (loose equality)
null === undefined → false (strict equality)
This is a special rule in JavaScript for loose equality (==).

3 EXAMPLES WITH EXPLANATION

Expression	Step by Step Explanation	Result
null == undefined	• null and undefined have a special equality rule in JavaScript. • They are considered equal only to each other in loose equality (==).	true
null == 0	• JavaScript does NOT convert null to 0 in == comparison. • null is only loosely equal to undefined. • Therefore null == 0 is false.	false
undefined == ""	• undefined is not converted to an empty string. • They are different values. • Therefore they are not equal.	false
null == false	• JavaScript does NOT convert null to 0 in == comparison. • null is only loosely equal to undefined. • Therefore null == false is false.	false
undefined == false	• undefined is only loosely equal to null. • It is not equal to false. • Therefore the result is false.	false

KEY TAKEAWAY
- In loose equality (==), null is only equal to undefined.
- null is NOT equal to any other value (0, false, "", etc.).
- undefined is only equal to null, not to any other value.

4 PRACTICAL CHECK

```
console.log(null == undefined);   // true
console.log(null === undefined);  // false
console.log(null == 0);           // false
console.log(undefined == 0);      // false
console.log(null == "");          // false
console.log(undefined == "");     // false
```

EXPLANATION
- null == undefined is true because == performs type coercion and treats both as equal.
- === checks both type and value, so null and undefined are not strictly equal.
- null and undefined are not equal to any other value (number, string, etc.).

Key Takeaway
== : checks type coercion + value
=== : checks type and value both

1 NULL COMPARISON BEHAVIOR IN JAVASCRIPT

LOOSE EQUALITY (==) vs RELATIONAL COMPARISONS (<, >, <=, >=)

```
console.log(null == 0);   // false
console.log(null < 0);    // false
console.log(null > 0);    // false
console.log(null <= 0);   // true
console.log(null >= 0);   // true
```

LOOSE EQUALITY (==)
- null is only loosely equal to undefined.
- It is NOT equal to any other value.
- null is NOT converted to 0 in == comparison.

null == undefined → true
null == 0 → false

RELATIONAL COMPARISONS
(<, >, <=, >=)
- For relational comparisons, null is converted to 0.
- Then normal numeric comparison happens.

null → 0
0 < 0 → false
0 > 0 → false
0 <= 0 → true
0 >= 0 → true

EXPLANATION
- In loose equality (==), null is only equal to undefined.
- In == comparison, null is NOT converted to 0.
- In relational comparisons (<, >, <=, >=), null is converted to 0.
- After conversion, normal numeric comparison is performed.
- Therefore: null < 0 and null > 0 are false.
- And null <= 0 and null >= 0 are true.

QUICK SUMMARY
Equality (==):
null == undefined → true
null == anything else → false
Relational (<, >, <=, >=):
null → 0, then compare with the number

2 UNDEFINED EQUALITY (==) CHECKS

```
console.log(undefined == 0);   // false
console.log(undefined < 0);    // false
console.log(undefined > 0);    // false
console.log(undefined <= 0);   // false
console.log(undefined >= 0);   // false
```

undefined is not converted to any number. So, every comparison results in false.

EXPLANATION
- ✓ undefined is not converted to any number.
- ✓ So, undefined compared with 0 will always be false.
- ✓ That is why all comparisons using == with undefined are false.

EQUALITY MEMORIZATION CASES

[] + []	{} + {}	{} + []	[] == false	"" == false	null == undefined	null === undefined
""	"[object Object]"	"[object Object]"	true	true	true	false

Boolean("false")	[] == ![]	"0" == false	0 == false	0 === false	NaN == NaN	NaN === NaN
true	true	true	true	false	false	false

Number("")	Number(" ")	Boolean("0")	Boolean([])
0	0	true	true

IMPORTANT POINTS
- JS converts values to make operations possible.
- + behaves differently from other operators.
- == allows conversion, === does not.
- Falsy values become false in conditions.
- You should know special coercion cases.

COMMON MISTAKES
- Using == instead of ===
- Assuming "0" is false (it's true)
- Forgetting NaN is not equal to itself
- Relying on implicit conversion too much
- Not remembering [] + {} cases

PRO TIPS
- ✓ Use === for safe comparison
- ✓ Convert values explicitly when possible
- ✓ Remember falsy list
- ✓ Practice special cases
- ✓ Understand ToPrimitive concept

REVISION SUMMARY
① JS has Implicit & Explicit Conversion
② Know type ↔ type conversion rules
③ Understand operator behavior
④ Learn equality comparison deeply
⑤ Master special coercion cases

MASTER TYPE CONVERSION →
WRITE BETTER,
BUG-FREE JAVASCRIPT!

> "When JavaScript converts types, it's not magic.
> It's logic — hidden, but predictable."

JAVASCRIPT TYPE CONVERSION & EQUALITY

MCQ QUESTIONS

01 `console.log(null == undefined);`
A true
B false
C NaN
D Error

02 `console.log(null === undefined);`
A true
B false
C NaN
D Error

03 `console.log(null >= 0);`
A true
B false
C NaN
D Error

04 `console.log(null > 0);`
A true
B false
C NaN
D Error

05 `console.log(null <= 0);`
A true
B false
C NaN
D Error

06 `console.log(undefined >= 0);`
A true
B false
C NaN
D Error

07 `console.log(undefined == 0);`
A true
B false
C NaN
D Error

08 `console.log(Number(null));`
A 0
B null
C NaN
D undefined

09 `console.log(Number(undefined));`
A 0
B undefined
C NaN
D Error

10 `console.log(Boolean(null));`
A true
B false
C null
D Error

11 `console.log(Boolean("false"));`
A true B false C NaN D Error

12 `console.log(Boolean("0"));`
A true B false C NaN D Error

13 `console.log(Boolean([]));`
A true B false C NaN D Error

14 `console.log(Boolean({}));`
A true B false C NaN D Error

15 `console.log("" == false);`
A true B false C NaN D Error

16 `console.log("0" == false);`
A true B false C Error D NaN

17 `console.log(0 == false);`
A true B false C Error D NaN

18 `console.log(0 === false);`
A true B false C Error D NaN

19 `console.log("5" - 2);`
A "52" B 52 C 3 D Error

20 `console.log("5" + 2);`
A 7 B "52" C 3 D Error

21 `console.log([] == false);`
A true B false C Error D NaN

22 `console.log([] == ![]);`
A true B false C Error D NaN

23 `console.log(NaN == NaN);`
A true B false C Error D NaN

24 `console.log(NaN === NaN);`
A true B false C Error D NaN

25 `console.log(Number(""));`
A NaN B 0 C "" D undefined

26 `console.log(Number(" "));`
A NaN B 0 C undefined D Error

27 `console.log([] + []);`
A [] B "" C 0 D Error

28 `console.log([] + {});`
A "[object Object]" B "{}" C Error D NaN

29 `console.log({} + []);`
A "[object Object]" B "{}" C Error D NaN

30 `console.log(true + true + false);`
A 3 B 2 C 1 D true

JAVASCRIPT TYPE CONVERSION & EQUALITY

ANSWERS WITH EXPLANATION (1 - 10)

01 `console.log(null == undefined);`
Answer
A true
Explanation
- In loose equality (==), null and undefined are considered equal to each other.
- Therefore → true.

02 `console.log(null === undefined);`
Answer
B false
Explanation
- Strict equality (===) checks both value and type.
- null (object type) and undefined (undefined type) are different types.
- Therefore → false.

03 `console.log(null >= 0);`
Answer
A true
Explanation
- In relational comparisons, null is converted to 0.
- So, 0 >= 0 → true.

04 `console.log(null > 0);`
Answer
B false
Explanation
- null is converted to 0.
- So, 0 > 0 → false.

05 `console.log(null <= 0);`
Answer
A true
Explanation
- null is converted to 0.
- So, 0 <= 0 → true.

06 `console.log(undefined >= 0);`
Answer
B false
Explanation
- undefined is converted to NaN in numeric context.
- Any comparison with NaN (except !=) is false.
- Therefore → false.

07 `console.log(undefined == 0);`
Answer
B false
Explanation
- undefined is converted to NaN in numeric context.
- NaN == 0 → false.

08 `console.log(Number(null));`
Answer
A 0
Explanation
- Number(null) converts null to 0.

09 `console.log(Number(undefined));`
Answer
C NaN
Explanation
- Number(undefined) converts undefined to NaN.

10 `console.log(Boolean(null));`
Answer
B false
Explanation
Boolean(null) → false because null is a falsy value.

KEY CONVERSIONS & RULES

NaN → Not equal to anything, even NaN (both == and ===)

Falsy Values:
false, 0, -0, 0n, "", null, undefined, NaN

11 `console.log(Boolean("false"));`
Answer
A true
Explanation
- "false" is a non-empty string.
- All non-empty strings are truthy.
- Result → true

12 `console.log(Boolean("0"));`
Answer
A true
Explanation
- "0" is a non-empty string.
- All non-empty strings are truthy.
- Result → true

13 `console.log(Boolean([]));`
Answer
A true
Explanation
- [] is an array.
- Arrays are objects in JavaScript.
- All objects are truthy.
- Result → true

14 `console.log(Boolean({}));`
Answer
A true
Explanation
- {} is an object.
- All objects are truthy.
- Result → true

15 `console.log("" == false);`
Answer
A true
Explanation
- false is converted to 0.
- "" is converted to 0.
- 0 == 0
- Result → true

16 `console.log("0" == false);`
Answer
A true
Explanation
- false is converted to 0.
- "0" is converted to 0.
- 0 == 0
- Result → true

17 `console.log(0 == false);`
Answer
A true
Explanation
- false is converted to 0.
- 0 == 0
- Result → true

18 `console.log(0 === false);`
Answer
B false
Explanation
- === checks both type and value.
- 0 is a number.
- false is a boolean.
- Different types.
- Result → false

19 `console.log("5" - 2);`
Answer
C 3
Explanation
- The - operator requires numeric operands.
- "5" is converted to 5.
- 5 - 2
- Result → 3

20 `console.log("5" + 2);`
Answer
B "52"
Explanation
- When one operand is a string, + performs string concatenation.
- 2 becomes "2".
- "5" + "2"
- Result → "52"

21 `console.log([] == false);`
Answer
A true
Explanation
- [] → ""
- false → 0
- "" → 0
- 0 == 0
- Result → true

22 `console.log([] == ![]);`
Answer
A true
Explanation
- [] → ""
- ![] → false
- false → 0
- "" → 0
- 0 == 0
- Result → true

23 `console.log(NaN == NaN);`
Answer
B false
Explanation
- NaN is not equal to anything, including itself.
- NaN == NaN
- Result → false

24 `console.log(NaN === NaN);`
Answer
B false
Explanation
- Even strict equality does not consider NaN equal to itself.
- NaN === NaN
- Result → false

25 `console.log(Number(""));`
Answer
B 0
Explanation
- Number("") converts an empty string to the number 0.
- Result → 0

26 `console.log(Number(" "));`
Answer
B 0
Explanation
- Whitespace is trimmed first.
- Number(" ") becomes Number("").
- Number("") returns 0.
- Result → 0

27 `console.log([] + []);`
Answer
B ""
Explanation
- [] → ""
- [] → ""
- "" + ""
- Result → ""

28 `console.log([] + {});`
Answer
A "[object Object]"
Explanation
- [] → ""
- {} → "[object Object]"
- "" + "[object Object]"
- Result → "[object Object]"

29 `console.log({} + []);`
Answer
A "[object Object]"
Explanation
- {} → "[object Object]"
- [] + ""
- "[object Object]" + ""
- Result → "[object Object]"

30 `console.log(true + true + false);`
Answer
B 2
Explanation
- true → 1
- true → 1
- false → 0
- 1 + 1 +
- Result → 2